



# 資料結構

## Data Structure

### Lab 10

姓名: 盧敬恩  
學號: 114310123

## Lab10-Ex1. (Heap)

### 執行Heapify

### Code

```
#include <iostream>
#include <vector>
#include <string>
#include <fstream>
#include <sstream>
using namespace std;
vector<int> readFromFile(const string& filename) { //這裡是讀取input檔案的程式
    vector<int> arr;
    ifstream file(filename);
    if (!file.is_open()) {
        cerr << "【錯誤】找不到檔案: " << filename << endl;
        return arr;
    }
    string line;
    if (getline(file, line)) {
        stringstream ss(line);
        string token;
        while (getline(ss, token, ',')) {
            if (!token.empty()) {
                arr.push_back(stoi(token));
            }
        }
    }
    file.close();
    return arr;
}
class MaxHeap {
public:
    vector<int> heap;
    void buildMaxHeap(vector<int>& arr) {
        heap = arr;
        for (int i = (heap.size() / 2) - 1; i >= 0; i--) {
            //這段是從TREE的葉子開始 往上做比較的判別
            heapify(i);
        }
    }
    void heapify(int i) {
        int largest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2; //這裡是完整的binary tree 左子樹 = 2i+1 右=2i+2
        if (left < heap.size() && heap[left] > heap[largest]) {
            largest = left; //如果左tree比當前parent 節點大 且左>右 largest 設為左tree的值
        }
        if (right < heap.size() && heap[right] > heap[largest]) {
            largest = right; //如果右tree比當前parent 節點大 且左<右 largest 設為右tree的值
        }
        if (largest != i) { //如果發生兩種其中一種讓最大的不再是parent節點 就交換
            swap(heap[i], heap[largest]);
            heapify(largest);
        }
    }
}
```

```

void printHeap() {
    for (int val : heap) {
        cout << val << " ";
    }
    cout << endl;
}

};

class MinHeap {
public:
    vector<int> heap;

    void buildMinHeap(vector<int>& arr) {
        heap = arr;
        for (int i = (heap.size() / 2) - 1; i >= 0; i--) {//這段是從TREE最後一個有小孩的節點開始 往上做比較的判別
            heapify(i);
        }
    }

    void heapify(int i) {
        int smallest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;//這裡是完整的binary tree 左子樹 = 2i+1 右=2i+2

        if (left < heap.size() && heap[left] < heap[smallest]) {
            smallest = left;//如果左tree比當前parent 節點小 且左<右 smallest 設為左tree的值
        }
        if (right < heap.size() && heap[right] < heap[smallest]) {
            smallest = right;//如果右tree比當前parent 節點小 且左>右 smallest 設為右tree的值
        }
        if (smallest != i) {//如果發生兩種其中一種讓最小的不再是parent節點 就交換
            swap(heap[i], heap[smallest]);
            heapify(smallest);
        }
    }

    void printHeap() {
        for (int val : heap) {
            cout << val << " ";
        }
        cout << endl;
    }
};

int main() {
    string filename = "ww.txt";
    vector<int> arr = readFromFile(filename);
    if (arr.empty()) {
        cout << "\n讀取失敗，請確認 ww.txt 內是否有正確的資料。" << endl;
        system("pause");
        return -1;
    }
    cout << "Input Array: ";
    for (int val : arr) {
        cout << val << " ";
    }
    cout << endl << endl;
    MaxHeap maxHeap;

```

```

maxHeap.buildMaxHeap(arr);
cout << "Max Heap: ";
maxHeap.printHeap();
cout << endl;
MinHeap minHeap;
minHeap.buildMinHeap(arr);
cout << "Min Heap: ";
minHeap.printHeap();
cout << endl;
system("pause");
return 0;
}

```

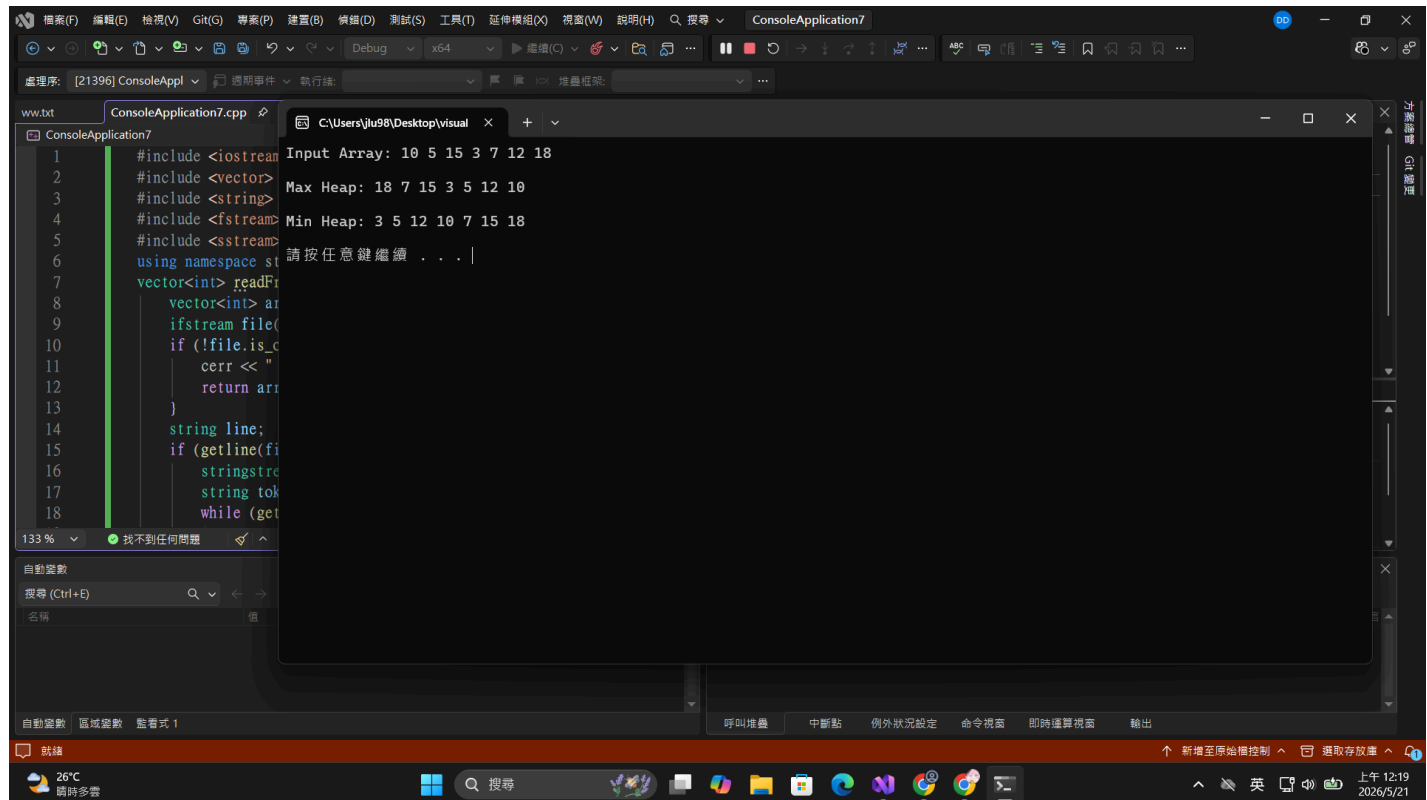
## Result

INPUT1:

```

C:\Users\jlu98\Desktop\visual
Input Array: 10 5 15 3 7 12 18
Max Heap: 18 7 15 3 5 12 10
Min Heap: 3 5 12 10 7 15 18
請按任意鍵繼續 . . . |

```



INPUT2:

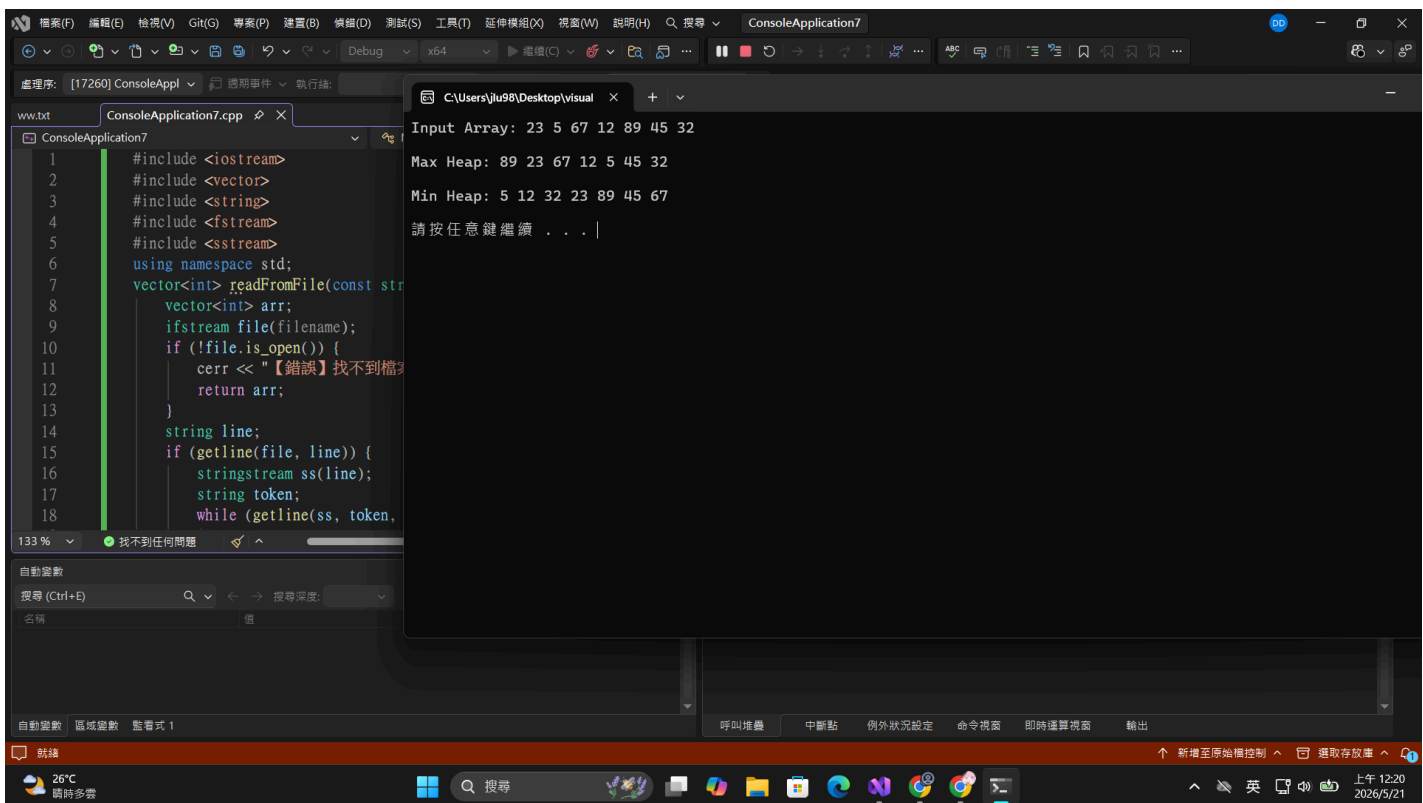
```
C:\Users\jlu98\Desktop\visual x + v

Input Array: 23 5 67 12 89 45 32

Max Heap: 89 23 67 12 5 45 32

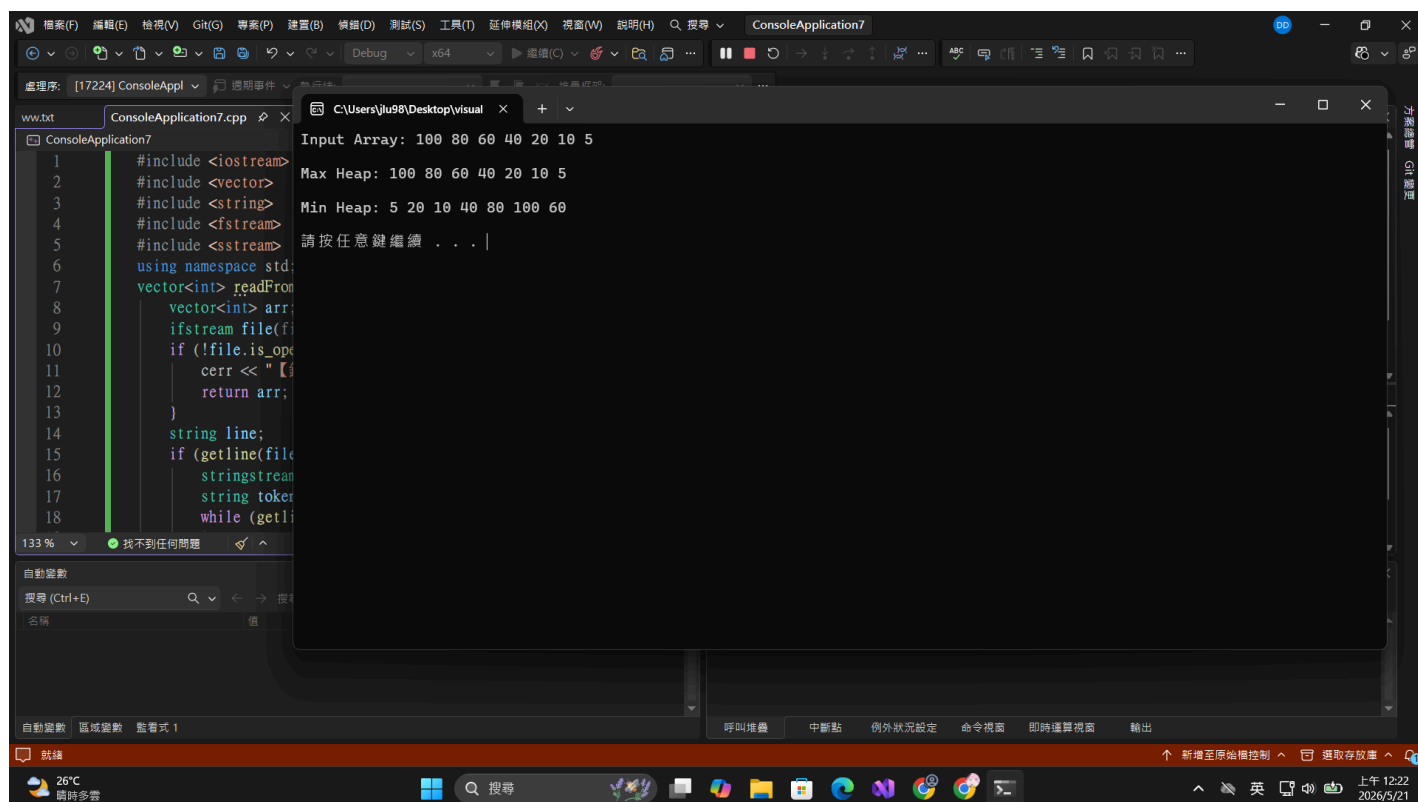
Min Heap: 5 12 32 23 89 45 67

請按任意鍵繼續 . . .
```



INPUT3:

```
C:\Users\jlu98\Desktop\visual × + ∨  
  
Input Array: 100 80 60 40 20 10 5  
  
Max Heap: 100 80 60 40 20 10 5  
  
Min Heap: 5 20 10 40 80 100 60  
  
請按任意鍵繼續 . . .
```



## Discussion

完成input 1 的max heap 和 min heap

